

**LAPORAN PRAKTIKUM**  
**ALGORITMA PEMROGRAMAN 2**  
**MODUL 12 – SEARCHING**



**ANNISA BERLIANA NINDYA SYAH PUTRI**

**109082500166**

**KELAS S1IF-13-05**

**PROGRAM STUDI TEKNIK INFORMATIKA**  
**FAKULTAS INFORMATIKA**  
**TELKOM UNIVERSITY PURWOKERTO**

**2026**

## A. Dasar Teori

### 1. Pengantar Pencarian Data (Searching)

Pencarian data (searching) merupakan proses untuk menemukan suatu data tertentu di dalam sekumpulan data yang biasanya disimpan dalam bentuk array. Berbeda dengan pencarian nilai maksimum atau minimum yang pasti menghasilkan nilai, pencarian data spesifik belum tentu berhasil karena data yang dicari bisa saja tidak terdapat dalam kumpulan tersebut.

Dalam implementasi program, hasil dari proses pencarian tidak hanya menunjukkan apakah data tersebut ada atau tidak, tetapi juga biasanya mengembalikan posisi atau indeks dari data tersebut di dalam array. Jika data tidak ditemukan, umumnya digunakan nilai khusus seperti -1 sebagai penanda.

### 2. Metode Pencarian Sekuensial (Sequential Search)

Pencarian sekuensial atau *linear search* adalah metode pencarian yang dilakukan dengan cara memeriksa setiap elemen dalam array secara berurutan, dimulai dari indeks pertama hingga terakhir.

Metode ini akan berhenti ketika data yang dicari sudah ditemukan, meskipun masih ada elemen lain yang belum diperiksa. Jika sampai elemen terakhir data tidak ditemukan, maka proses pencarian akan berakhir.

Keunggulan dari metode ini adalah dapat digunakan pada semua kondisi data, baik yang belum terurut maupun yang sudah terurut. Implementasinya juga cukup sederhana karena hanya menggunakan perulangan untuk membandingkan setiap elemen dengan nilai yang dicari.

### 3. Metode Pencarian Biner (Binary Search)

Pencarian biner merupakan metode pencarian yang lebih efisien dibandingkan pencarian sekuensial, namun memiliki syarat bahwa data harus dalam keadaan terurut.

Cara kerja metode ini adalah dengan membagi ruang pencarian menjadi dua bagian secara berulang. Pertama, ditentukan indeks tengah dari array, kemudian nilai tengah tersebut dibandingkan dengan data yang dicari.

Jika nilai tengah sama dengan data yang dicari, maka pencarian selesai. Jika tidak, maka pencarian dilanjutkan hanya pada setengah bagian array yang memungkinkan. Proses ini terus dilakukan hingga data ditemukan atau hingga batas pencarian tidak valid lagi.

Perlu diperhatikan bahwa arah pengurutan data sangat mempengaruhi proses binary search. Jika data terurut menaik (ascending), maka logika pencariannya berbeda dengan data yang terurut menurun (descending). Oleh karena itu, algoritma harus disesuaikan dengan kondisi data.

### 4. Pencarian pada Array dengan Tipe Data Terstruktur (Struct)

Pencarian juga dapat dilakukan pada array yang berisi data bertipe struct, seperti data mahasiswa yang memiliki beberapa field (misalnya nama, NIM, dan IPK).

Secara umum, prinsip pencariannya sama seperti pada tipe data dasar, namun yang dibandingkan adalah nilai dari field tertentu yang dijadikan sebagai kunci pencarian.

Untuk pencarian sekuensial, proses dapat dilakukan langsung tanpa memperhatikan urutan data. Sedangkan pada pencarian biner, data harus sudah terurut berdasarkan field yang digunakan sebagai acuan pencarian. Jika tidak sesuai, maka hasil pencarian bisa tidak valid.

#### A. Guided

##### 1. Program Pencarian Data Buah dengan Sequential Search

```
package main

import "fmt"

func SequentialSearch(arrBuah [5]string, dataDicari string) int {
    idx := -1
    for i := 0; i < len(arrBuah); i++ {
        if arrBuah[i] == dataDicari {
            idx = i
            break
        }
    }
    return idx
}

func main() {
    var arrBuah [5]string

    for i := 0; i < len(arrBuah); i++ {
        fmt.Printf("Masukkan data buah indeks ke-%d: ", i)
        fmt.Scan(&arrBuah[i])
    }

    var dataCari string
    fmt.Println("Masukkan data buah yang ingin dicari:")
    fmt.Scan(&dataCari)

    index_data := SequentialSearch(arrBuah, dataCari)
```

```

    if index_data != -1 {
        fmt.Printf("Data %s ditemukan pada indeks ke-%d!", dataCari,
index_data)
    } else {
        fmt.Printf("Data %s tidak ditemukan!", dataCari)
    }
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11>
● go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\We
ek 11\Guided\Guided-1\guided1.go"
Masukkan data buah indeks ke-0: apel
Masukkan data buah indeks ke-1: pir
Masukkan data buah indeks ke-2: anggur
Masukkan data buah indeks ke-3: jeruk
Masukkan data buah indeks ke-4: melon
Masukkan data buah yang ingin dicari:
jeruk
Data jeruk ditemukan pada indeks ke-3!

```

## 2. Program Pencarian Data dengan Binary Search

```

package main

import "fmt"

const max = 100

type arr [max]int

func BinarySearch(a arr, n int, X int) bool {
    var kiri int = 0
    var kanan int = n - 1
    var found bool = false
    var tengah int

    //ascending
    for kiri <= kanan && !found {
        tengah = (kiri + kanan) / 2
        if a[tengah] < X {
            kiri = tengah + 1
        } else if a[tengah] > X {
            kanan = tengah - 1
        } else {
            found = true
        }
    }
}

```

```

    return found
}

func main() {
    var data arr
    var n, X int

    fmt.Print("Input jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Input data ascending")
    for i := 0; i <= n; i++ {
        fmt.Scanln(&data[i])
    }

    fmt.Println("Data yang dicari: ")
    fmt.Scan(&X)

    if BinarySearch(data, n, X) {
        fmt.Println("Data ditemukan")
    } else {
        fmt.Println("Data TIDAK ditemukan")
    }
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11>
● go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\We
ek 11\Guided\Guided-2\guided2.go"
Input jumlah data: 5
Input data ascending
1
2
3
4
5
Data yang dicari:
3
Data ditemukan

```

### 3. Program Pencarian Data Mahasiswa Menggunakan Sequential dan Binary Search

```

package main

import "fmt"

type mahasiswa struct {

```

```

    nama string
    nim int
    IPK float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim { //kurang dari median,
pencarian ke kiri
            kn = med - 1
        } else if nimDicari > arrData[med].nim { //Lebih dari median,
pencarian ke kanan
            kr = med + 1
        } else {
            found = med
        }
    }
    return found
}

func seqSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var mahasiswa06 arrMahasiswa
    var nimDicari int

    fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---")
    for i := 0; i < len(mahasiswa06); i++ {

```

```

        fmt.Printf("=== Masukkan data mahasiswa 06 pada indeks ke-%d
===\n", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&mahasiswa06[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&mahasiswa06[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&mahasiswa06[i].IPK)
        fmt.Println("-----")
    }
    fmt.Println()

    fmt.Print("Masukkan NIM mahasiswa yang ingin dicari : ")
    fmt.Scan(&nimDicari)
    fmt.Println()

    var idxHasilCariSeqSearch int
    idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06, nimDicari)

    fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
    if idxHasilCariSeqSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di
indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariSeqSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariSeqSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariSeqSearch].IPK)
    } else if idxHasilCariSeqSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada
array!", nimDicari)
    }
    fmt.Println()

    var idxHasilCariBinSearch int
    idxHasilCariBinSearch = binSearchStruct(mahasiswa06, nimDicari)

    fmt.Println("== HASIL BINARY SEARCH ==")
    if idxHasilCariBinSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di
indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
        fmt.Println("Berikut Detail Datanya : ")
    }

```

```

        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariBinSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariBinSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariBinSearch].IPK)
    } else if idxHasilCariBinSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada
array!", nimDicari)
    }
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11> go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\10908
2500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11\Guided\Guided-3\guided3.go"
--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---
=== Masukkan data mahasiswa 06 pada indeks ke-0 ===
Masukkan nama : dhimas
Masukkan NIM : 111
Masukkan IPK : 3.21
-----
=== Masukkan data mahasiswa 06 pada indeks ke-1 ===
Masukkan nama : hafizh
Masukkan NIM : 222
Masukkan IPK : 3.27
-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : fathur
Masukkan NIM : 333
Masukkan IPK : 2.19
-----
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : rahman
Masukkan NIM : 444
Masukkan IPK : 3
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : regita
Masukkan NIM : 555
Masukkan IPK : 3.74
-----

Masukkan NIM mahasiswa yang ingin dicari : 333

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 333 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : fathur
NIM : 333
IPK : 2.19

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 333 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : fathur
NIM : 333
IPK : 2.19

```

## B. Unguided

### 1. Soal Unguided 1

Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau



dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT.

Buatlah program **pilkart** yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.

**Masukan** hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah integer dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

**Keluaran** dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian sejumlah baris yang mencetak data para calon apa saja yang mendapatkan suara.

| No | Masukan                    | Keluaran                                                                          |
|----|----------------------------|-----------------------------------------------------------------------------------|
| 1  | 7 19 3 2 78 3 1 -3 18 19 0 | Suara masuk: 10<br>Suara sah: 8<br>1: 1<br>2: 1<br>3: 2<br>7: 1<br>18: 1<br>19: 2 |

Jawaban :

```
package main

import "fmt"

func main() {
    var x int
    total := 0
    sah := 0

    var hitung [21]int

    for {
        fmt.Scan(&x)

        if x == 0 {
            break
        }

        total++
    }
```

```

        if x >= 1 && x <= 20 {
            hitung[x] = hitung[x] + 1
            sah++
        }
    }

    fmt.Println("Suara masuk:", total)
    fmt.Println("Suara sah:", sah)

    for i := 1; i <= 20; i++ {
        if hitung[i] != 0 {
            fmt.Println(i, ":", hitung[i])
        }
    }
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11>
go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11\Unguided\Unguided-1\unguided1.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1 : 1
2 : 1
3 : 2
7 : 1
18 : 1
19 : 2

```

Penjelasan :

Program ini membaca input angka satu per satu sampai ketemu 0 sebagai tanda berhenti. Setiap angka yang masuk tetap dihitung sebagai suara masuk.

Lalu program mengecek apakah angka tersebut berada di antara 1 sampai 20. Kalau iya, berarti dianggap suara sah dan jumlahnya disimpan di array sesuai nomor calon.

Di akhir, program menampilkan total semua suara yang masuk, jumlah suara yang valid, lalu daftar calon yang mendapatkan suara beserta jumlahnya.

## 2. Soal Unguided 2

Berdasarkan program sebelumnya, buat program **pilkart** yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang sama, ketua terpilih adalah dengan

nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

**Masukan** hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah bilangan bulat dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

**Keluaran** dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian tercetak calon nomor berapa saja yang menjadi pasangan ketua RT dan wakil ketua RT yang baru.

| No | Masukan                    | Keluaran                                                          |
|----|----------------------------|-------------------------------------------------------------------|
| 1  | 7 19 3 2 78 3 1 -3 18 19 0 | Suara masuk: 10<br>Suara sah: 8<br>Ketua RT: 3<br>Wakil ketua: 19 |

Jawaban :

```
package main

import "fmt"

func main() {
    var x int
    total := 0
    sah := 0

    var hitung [21]int

    for {
        fmt.Scan(&x)

        if x == 0 {
            break
        }

        total++

        if x >= 1 && x <= 20 {
            hitung[x]++
            sah++
        }
    }

    ketua := 0
```

```

wakil := 0

for i := 1; i <= 20; i++ {
    if hitung[i] > hitung[ketua] {
        wakil = ketua
        ketua = i
    } else if hitung[i] > hitung[wakil] && i != ketua {
        wakil = i
    }
}

fmt.Println("Suara masuk:", total)
fmt.Println("Suara sah:", sah)
fmt.Println("Ketua RT:", ketua)
fmt.Println("Wakil ketua:", wakil)
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11>
● go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11\Unguided\Unguided-2\unguided2.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19

```

Penjelasan :

Program ini membaca data suara satu per satu sampai ketemu angka 0 sebagai tanda selesai. Semua data tetap dihitung sebagai suara masuk, tapi yang dihitung sebagai suara sah hanya yang nilainya 1 sampai 20.

Setiap suara sah disimpan dan dihitung jumlahnya di array sesuai nomor calon.

Setelah itu, program mencari siapa yang punya suara paling banyak sebagai ketua, dan yang terbanyak kedua sebagai wakil. Prosesnya dilakukan dengan membandingkan jumlah suara tiap calon.

Terakhir, program menampilkan jumlah suara masuk, jumlah suara sah, lalu hasil ketua dan wakil yang terpilih.

### 3. Soal Unguided 3

Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k, apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

**Masukan** terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu n dan k. n menyatakan banyaknya data, dimana  $1 < n \leq 1000000$ . k adalah bilangan yang ingin dicari. Baris kedua berisi n buah data integer positif yang sudah terurut membesar.

**Keluaran** terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari (k) dalam kumpulan data yang diberikan. Posisi data dihitung dimulai dari angka 0. Atau memberikan keluaran "TIDAK ADA" jika data k tersebut tidak ditemukan dalam kumpulan. Program yang dibangun harus menggunakan subprogram dengan mengikuti kerangka yang sudah diberikan berikut ini.

```
package main
import "fmt"

const NMAX = 1000000
var data [NMAX]int

func main() {
    /* buatlah kode utama yang membaca baris pertama (n dan k). kemudian data
    diisi
    oleh prosedur isiArray(n), dan pencarian oleh fungsi posisi(n,k), dan
    setelah
    itu output dicetak. */
}

func isiArray(n int) {
    /* I.S. terdefinisi integer n, dan sejumlah n data sudah siap pada piranti
    masukan.
    F.S. Array data berisi n (<=NMAX) bilangan */
}

func posisi(n, k int) int {
    /* mengembalikan posisi k dalam array data dengan n elemen. Posisi dimulai
    dari
    posisi 0. Jika tidak ada kembalikan -1 */
}
```

Contoh masukan dan keluaran:

| No | Masukan                                           | Keluaran  | Penjelasan                                                |
|----|---------------------------------------------------|-----------|-----------------------------------------------------------|
| 1  | 12 534<br>1 3 8 16 32 123 323 323 534 543 823 999 | 8         | Data 534 berada pada posisi ke-8 dihitung dari awal data. |
| 2  | 12 535<br>1 3 8 16 32 123 323 323 534 543 823 999 | TIDAK ADA |                                                           |

Jawaban :

```
package main
```

```

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func main() {
    var n, k int
    fmt.Scan(&n, &k)

    isiArray(n)

    idx := posisi(n, k)

    if idx == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(idx)
    }
}

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    kiri := 0
    kanan := n - 1

    for kiri <= kanan {
        tengah := (kiri + kanan) / 2

        if data[tengah] == k {
            return tengah
        } else if data[tengah] < k {
            kiri = tengah + 1
        } else {
            kanan = tengah - 1
        }
    }

    return -1
}

```

Output :

```
PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11>
go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11\Unguided\Unguided-3\unguided3.go"
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11>
go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 11\Unguided\Unguided-3\unguided3.go"
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA
```

Penjelasan :

Program ini membaca jumlah data dan nilai yang ingin dicari, lalu mengisi array dengan data yang sudah terurut.

Setelah itu, pencarian dilakukan menggunakan metode binary search dengan membagi ruang pencarian menjadi dua bagian secara terus-menerus sampai nilai ditemukan atau tidak ada lagi kemungkinan.

Jika data ditemukan, program menampilkan indeksinya. Kalau tidak ada, program menampilkan "TIDAK ADA".

### C. Kesimpulan

Kesimpulannya, praktikum ini menunjukkan bahwa pencarian data bisa dilakukan dengan dua cara, yaitu sequential search dan binary search. Sequential search memeriksa data satu per satu sehingga bisa dipakai untuk data yang belum terurut, tetapi lebih lambat jika datanya banyak.

Sementara itu, binary search lebih cepat karena membagi data menjadi dua bagian setiap langkah, namun hanya bisa digunakan jika data sudah dalam keadaan terurut.

Dari percobaan juga terlihat bahwa penggunaan array membantu proses pencarian dan perhitungan data jadi lebih terstruktur, terutama saat mengolah banyak data sekaligus seperti pada kasus pemilihan suara dan data mahasiswa.

